LAB8 – Cloud

1. Projeto CI/CD

Vamos explorar os recursos disponíveis e configurados:

FastAPI	Docker Hub	AWS EC2

Lembre-se que agora precisamos colocar em prática o domínio sobre alguns assuntos já estudados.

Recurso	Descrição / Onde Obter
Conta GitHub	Com o repositório criado. github.com
Conta Docker Hub	Para armazenar a imagem Docker. hub.docker.com
Conta AWS	Para provisionar a instância EC2. aws.amazon.com
Instância EC2 rodando	Amazon Linux 2 ou Ubuntu 22.04 LTS com Docker instalado
Python 3.11+	Instalado localmente para desenvolvimento
Docker Desktop	Para testar a imagem localmente antes do deploy
Git	Configurado com acesso ao repositório

Sobre a EC2

O guia assume que você possui uma instância EC2 em execução com:

- Docker instalado e serviço ativo (sudo systemctl start docker)
- Um usuário configurado para executar Docker sem sudo (grupo docker)
- Porta 8080 liberada no Security Group para acesso externo à API

2. Estrutura do Projeto

Organize seu repositório da seguinte forma para que os workflows funcionem corretamente:

```
minha-api/
├── .github/
│   └── workflows/
│       ├── docker-hub.yml   # Pipeline Docker Hub
│       └── ec2-deploy.yml    # Pipeline EC2
├── app/
```

```
│   ├── main.py             # FastAPI application
│
├── tests/
│   └── test_main.py        # Testes com pytest
├── Dockerfile
├── .dockerignore
├── requirements.txt
```

3. A API Python (FastAPI)

Nos exemplos a seguir usamos FastAPI, mas o mesmo pipeline funciona com Flask ou Django REST Framework.

3.1 requirements.txt

requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
pydantic==2.7.1
```

3.2 app/main.py

app/main.py

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Minha API", version="1.0.0")

class Item(BaseModel):
    name: str
    price: float

@app.get("/")
async def root():
    return {"status": "ok", "message": "API rodando!"}

@app.get("/health")
async def health():
    return {"status": "healthy"}

@app.post("/items")
async def create_item(item: Item):
    return {"created": item}
```

3.3 tests/test_main.py

tests/test_main.py

```python
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_root():
    res = client.get("/")
    assert res.status_code == 200
    assert res.json()["status"] == "ok"

def test_health():
    res = client.get("/health")
    assert res.status_code == 200
```

Para validar se o programa funciona, precisamos executar no desktop (ou na VM) que tenha a instalação do Python ativa.

Faça a instalação do ambiente isolado do app com as bibliotecas necessárias:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
pip install pytest
```

Entre na pasta **APP/** e execute:

```
python -m uvicorn main:app --reload --port 8080
```

```
INFO:     Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
INFO:     Started reloader process [40764] using WatchFiles
INFO:     Started server process [30428]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:58240 - "GET / HTTP/1.1" 200 OK
```

Agora vamos fazer o teste localmente.

```
python -m pytest tests/ -v --tb=short
```

```
rootdir: C:\Users\nisfleigaloni-ieg\Downloads\Senai_FastAPI-main\Senai_FastAPI-main
plugins: anyio-4.13.0
collected 2 items

tests/test_main.py::test_root PASSED
tests/test_main.py::test_health PASSED
```

4. Dockerfile

Use a imagem oficial Python slim para manter o tamanho da imagem reduzido. O padrão abaixo segue boas práticas de produção.

Dockerfile

```
# ── Stage 1: build deps ──────────────────────────────
FROM python:3.11-slim AS builder

WORKDIR /app

RUN pip install --upgrade pip
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ── Stage 2: runtime ─────────────────────────────────
FROM python:3.11-slim

WORKDIR /app

# Cria usuário não-root para segurança
RUN addgroup --system appgroup && adduser --system --ingroup appgroup appuser

COPY --from=builder /usr/local/lib/python3.11/site-packages
/usr/local/lib/python3.11/site-packages
COPY --from=builder /usr/local/bin/uvicorn /usr/local/bin/uvicorn
COPY app/ ./app/

USER appuser

EXPOSE 8080

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Voce pode implantar esse dockerfile no Desktop ou VM para validar se o container funciona corretamente.

> ⚠ **Atenção**
>
> Lembre-se que precisamos na primeira vez, fazer todos os passos de implantação nos servidores para posteriormente conseguir **AUTOMATIZAR**.

5. Configuração de Secrets no GitHub

Os workflows precisam de credenciais armazenadas de forma segura. Nunca coloque senhas ou chaves diretamente nos arquivos YAML.

5.1 Como adicionar Secrets

Acesse: GitHub → Repositório → Settings → Secrets and variables → Actions → New repository secret

5.2 Secrets necessários para o Pipeline 1 (Docker Hub)

Secret Name	Valor / Como Obter
DOCKERHUB_USERNAME	Seu nome de usuário no Docker Hub
DOCKERHUB_TOKEN	Gere em hub.docker.com → Account Settings → Security → Access Tokens

5.3 Secrets necessários para o Pipeline 2 (EC2)

Secret Name	Valor / Como Obter
DOCKERHUB_USERNAME	Mesmo do pipeline anterior
DOCKERHUB_TOKEN	Mesmo do pipeline anterior
EC2_HOST	IP público ou DNS da instância EC2 (ex: 54.200.10.30)
EC2_USERNAME	Usuário SSH da EC2 (ex: ec2-user ou ubuntu)
EC2_SSH_KEY	Conteúdo completo do arquivo .pem (inclua o cabeçalho BEGIN RSA PRIVATE KEY)

> ⚠ **Atenção com a SSH Key**
> Ao colar a chave SSH no secret, copie o conteúdo COMPLETO do arquivo .pem, incluindo as linhas -----BEGIN RSA PRIVATE KEY----- e -----END RSA PRIVATE KEY-----.
> Sem isso, a conexão SSH falhará com erro de autenticação.

6. Pipeline — Build

Este workflow é acionado em todo push para a branch main. Ele executa os testes, faz o build do APP para validar as funcionalidades

> **Fluxo do Pipeline**
> push → main → Checkout → Setup Python → Instalar deps → Executar testes

.github/workflows/docker-hub.yml

```yaml
# ─────────────────────────────────────────────────────
# Pipeline 1: Build e Test
# ─────────────────────────────────────────────────────
name: Builds and Tests

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]


jobs:

  # ── JOB 1: Testes ──────────────────────────────
  Build:
    name: Compilar Projeto
    runs-on: ubuntu-latest

    steps:
      - name: Checkout do código
        uses: actions/checkout@v4

      - name: Configurar Python
        uses: actions/setup-python@v3
```

```
        with:
          python-version: "3.11"

      - name: Instalar dependências
        run: |
          pip install --upgrade pip
          if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
          pip install pytest httpx  # deps de teste

      - name: Executar testes com pytest
        run: |
           PYTHONPATH=. pytest tests/ -v --tb=short
```

7. Pipeline — Deploy no Docker Hub

Este workflow é acionado em todo push para a branch main.

Fluxo do Pipeline Docker Hub
→ Login Docker Hub → Build imagem → Push com tag SHA + latest

.github/workflows/docker-hub.yml

```
# ──────────────────────────────────────────────────────────────

  Imagem:
    name: Construindo Imagem para Docker
    runs-on: ubuntu-latest
    needs: Build
    steps:
      - name: Login no Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}


        # Lembre-se de ajusta com o ID da sua conta no dockerhub
      - name: Build e Push Docker
        uses: docker/build-push-action@v7
        with:
          push: true
          tags: |
            ngaloni/senai_fastapi:latest
            ngaloni/senai_fastapi:${{ github.sha }}
```

```
      - name: Resumo do deploy
        run: |
           echo "## ☑ Imagem publicada no Docker Hub" >> $GITHUB_STEP_SUMMARY
           echo "Tags: ${{ steps.meta.outputs.tags }}" >> $GITHUB_STEP_SUMMARY
```

8. Pipeline — Deploy na AWS EC2

Este workflow é acionado após o build da imagem no Docker Hub. Ele conecta via SSH na instância EC2 e executa o pull da nova imagem, parando o container antigo e iniciando o novo.

Fluxo do Pipeline EC2

push → main → → SSH na EC2
→ docker pull (nova imagem) → docker stop (container atual)
→ docker run (novo container na porta 8080) → health check

.github/workflows/ec2-deploy.yml

```yaml
# ──────────────────────────────────────────────────────
# Pipeline : Deploy na instância AWS EC2
# ──────────────────────────────────────────────────────

Deploy:
    name: Implantação AWS
    runs-on: ubuntu-latest
    needs: Imagem

    steps:
      - name: Deploy via SSH na EC2
        uses: appleboy/ssh-action@v1.0.3
        with:
             host: ${{ secrets.EC2_HOST }}
             username: ${{ secrets.EC2_USERNAME }}
             key: ${{ secrets.EC2_SSH_KEY }}
             #envs: IMAGE_NAME,CONTAINER_NAME,APP_PORT
             script: |
                 # Faz pull da nova imagem
                 sudo docker pull docker.io/ngaloni/senai_fastapi:latest

                 # Para e remove o container atual (ignora erro se não existir)
                 sudo docker stop senai_fastapi 2>/dev/null || true
                 sudo docker rm senai_fastapi 2>/dev/null || true

                 # Sobe o novo container
                 sudo docker run -d \
                  --name senai_fastapi \
                  --restart unless-stopped \
                  -p 8080:8080 \
                  ngaloni/senai_fastapi:latest
```

```
# Health check pós-deploy
sleep 5
curl -f http://localhost:8080/health || exit 1
echo "☑ Deploy concluído com sucesso!"
# Remove imagens antigas (economiza espaço em disco)
sudo docker image prune -f
```

Dica: Rollback rápido

Para reverter para a versão anterior, execute no servidor EC2:

docker stop minha-api && docker rm minha-api

docker run -d --name minha-api -p 8080:8080 SEU_USER/minha-api:sha-COMMIT_ANTERIOR

8. Fluxo Completo End-to-End

Entenda o que acontece do git push até a API acessível ao usuário final:

Passo	O que acontece
1. git push → main	Desenvolvedor faz merge/push na branch main
2. Trigger do workflow	GitHub detecta o push e inicia o job test automaticamente
3. Checkout & Setup Python	Runner baixa o código e configura o ambiente Python 3.11
4. pip install + pytest	Dependências instaladas; todos os testes precisam passar (exit code 0)
5. docker login	Action autentica no Docker Hub usando os secrets
6. docker buildx build	Imagem multi-stage construída com cache do GHA (builds futuros mais rápidos)
7. docker push	Imagem publicada com tag :sha-XXXXXXX e :latest
8. SSH na EC2	appleboy/ssh-action conecta na instância via chave privada RSA
9. docker pull	EC2 baixa a nova imagem do Docker Hub
10. docker stop + rm	Container anterior é parado e removido sem downtime prolongado
11. docker run	Novo container sobe com --restart unless-stopped na porta 8080
12. Health check	curl /health valida que a API está respondendo antes de sinalizar sucesso
13. image prune	Imagens não utilizadas são removidas para liberar espaço em disco